Sport Management Simulation Game

Design and Architecture Document

Prepared by:

Şilan ATÇI	20230602008
Burak ÇALIŞKAN	20230602017
Beray KAYA	20230602045
Duru Eda CAN	20240602019

Instructor:
Doç. Dr. Kaya OĞUZ

Department of Computer Engineering

CE 216 Project

March 2026

Contents

1 Introduction and Problem Definition **2**
1.1 Project Objective 2
1.2 System Functionality 2

2 System Design and Architecture **3**
2.1 Layered Architecture 3
2.2 Match Simulation Sequence 4

3 Domain Model **5**
3.1 Overview 5
3.2 Core Entities 5
3.3 Domain Relationships 6

4 Sport Plugin Architecture **8**
4.1 Overview 8
4.2 Example Sport (Football) 8

5 GameFacade API **9**

6 Conclusion **10**

1 Introduction and Problem Definition

1.1 Project Objective

The main purpose of this project is to design a flexible Sports Manager infrastructure that can support different sports and where new sports can be easily integrated. The project aims to create an extensible framework without being dependent on a single sport.

1.2 System Functionality

The system is a flexible management application that allows the user to manage their own league by choosing between different sports. While the system manages basic processes such as team building, player listing, match simulation and point calculation, it also includes dynamic elements such as player injuries in the simulation. In terms of architecture, the user interface (UI) is designed to communicate only through defined interfaces, working independently of the technical details of sports. In this way, the system offers an orderly and expandable infrastructure where new sports can be easily integrated without disrupting the existing structure.

2 System Design and Architecture

2.1 Layered Architecture

In this project, layered architecture was used to make the system orderly and expandable. Each layer has a specific responsibility and does not assume the task of another layer. Thanks to this structure, the system is more understandable and suitable for the addition of new sports.

The system is divided into the following layers:

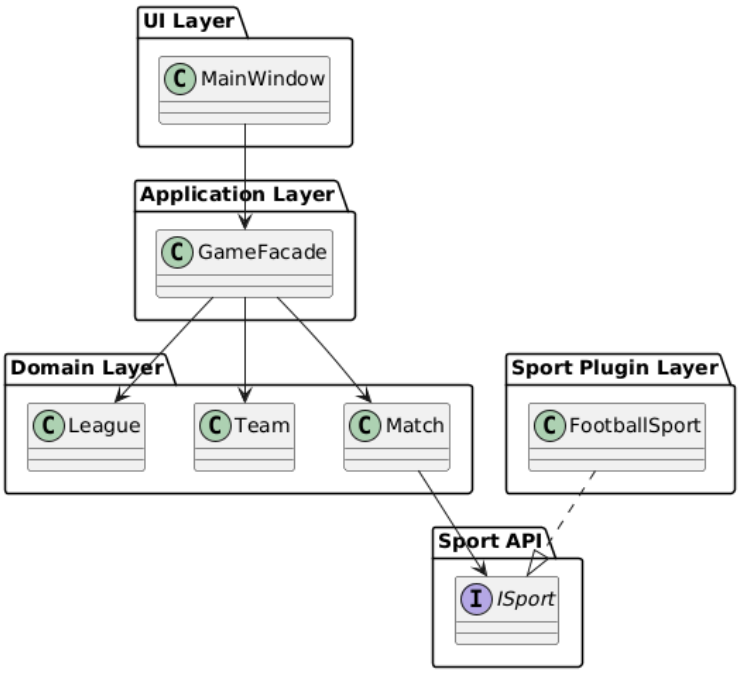

Figure 2.1: System Architecture Layers

- **UI Layer:** This layer provides interaction with the user. Screens, buttons, tables and entries from the user are located in this layer. The UI layer does not know the details of the sport. It only communicates with the system via the Application Layer. Thus, UI and business logic are separated from each other.

- **Application Layer:** This layer acts as a bridge between the UI and the Domain layer. It receives the operations from the user, calls the necessary domain objects and transmits the result back to the UI. This layer controls the overall flow of the system. UI does not directly access domain classes.

- **Domain Layer:** This layer contains the core logic of the system. Operations such as league structure, teams, players, match simulation and score calculation are performed here. This layer does not know the rules of sports directly, it only works through the Sport interface. Thus, the system remains independent of the type of sport.

- **Sport Plugin Layer:** This layer contains its own unique rules of sports. Each sport implements a common Sport interface. To add a sport, it is enough to write a new class that only implements the Sport interface. The current system code will not be changed.

2.2 Match Simulation Sequence

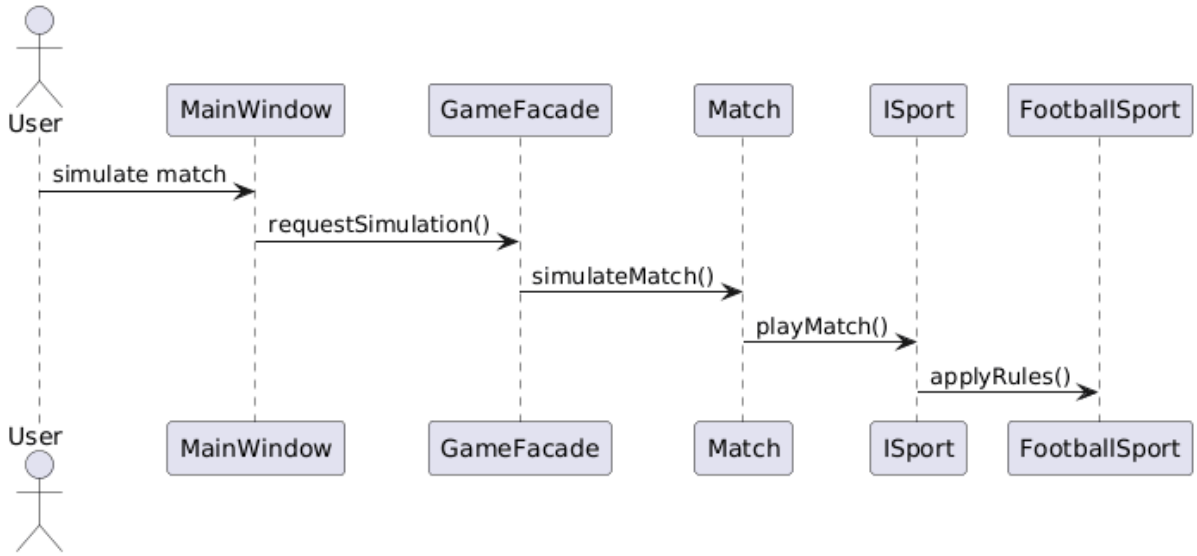

Figure 2.2: Match Simulation Sequence Diagram

This sequence diagram illustrates how a match simulation request flows through the system. The user interacts with the UI layer (MainWindow), which forwards the request to the GameFacade in the Application layer. The GameFacade coordinates the simulation by invoking the Match object in the Domain layer. The Match then calls the Sport API (ISport), which delegates the execution to the specific sport implementation (FootballSport).

3 Domain Model

3.1 Overview

The domain model represents the main structure of our sports management system. It includes the core entities such as Season, League, Team, Player, Fixture, and Match. These classes define how a sports season is organized and how different parts of the system are connected to each other.

The domain layer is designed to be sport independent. This means that it does not include any specific rules of sports like scoring systems or match flow details. Instead, it only contains the general structure that is common for all sports. The goal is to create a reusable core model that can work with different sports.

Specific features of sports, such as how points are calculated or how matches are played, are handled separately in the Sport Plugin layer. The domain layer only interacts with abstract sport interfaces, not with concrete sport implementations. This allows us to add new sports in the future without changing the existing domain classes. In this way, the system becomes more modular and easier to extend.

3.2 Core Entities

- **Season:** Season represents a sports season in which competitions take place. A season may contain multiple leagues and defines the time period in which matches are played.
- **League:** League represents a competition where teams participate during a season. It manages teams and the fixture of matches.
- **LeagueTable:** LeagueTable represents the standings of teams within a league. It keeps track of the performance of each team based on match results, including points and rankings throughout the season.
- **Team:** Team represents a sports team participating in a league. A team contains multiple players and competes against other teams in matches.
- **StandingEntry:** StandingEntry represents the position and statistical record of a specific team in the league table. It stores information such as the team, points earned, and other statistics used to determine rankings.

- **Player:** Player represents an individual athlete who belongs to a team and participates in matches.
- **Fixture:** Fixture represents the schedule of matches that will be played during a season. It organizes matches in a structured way within the league.
- **Match:** Match represents a game played between two teams. The match result affects the league standings, while specific match rules of sport are handled by the Sport Plugin layer.
- **Round (MatchWeek):** Round represents a group of matches that are played within the same week or stage of the competition. It organizes matches in the fixture so that teams play according to the scheduled competition rounds.
- **MatchResult:** MatchResult represents the outcome of a match between two teams. It records the final score and determines how the result affects the league standings.
- **Coach:** Coach represents a staff member responsible for training and managing players within a team. Coaches support the development of players and may influence team preparation during the season.

3.3 Domain Relationships

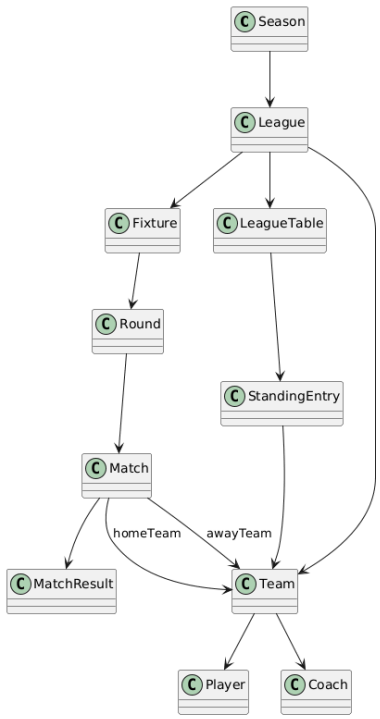

Figure 3.1: Domain Model Relationships

The diagram above shows the relationships between the core domain entities of the system. These relationships define how different parts of a sports season are connected to each other and represent the structural hierarchy of a sports competition.

A Season contains one or more leagues. Each League includes multiple teams and manages both a fixture and a league table. Teams exist independently in the league, while matches reference the participating teams as home and away teams. The Fixture represents the schedule of matches and is organized into rounds (or match weeks), where each Round contains multiple matches. Each Match is played between two teams and produces a MatchResult that records the outcome of the game.

A Team consists of multiple players and may also include coaches who support the team during the season. Each Player belongs to a single team. The LeagueTable stores the standings of teams in the league, where each StandingEntry represents the ranking and performance of a specific team.

4 Sport Plugin Architecture

4.1 Overview

The system supports different sports by using a plugin structure. Instead of putting sport rules inside the main system, each sport is created as a separate plugin. This keeps the main system independent from any specific sport. All sport-related rules are defined through a common set of interfaces called the Sport API. The Sport API defines what a sport must provide, such as how a match is played, how points are calculated, how ties are handled, and how teams are formed. Each sport plugin implements these rules in its own way. With this design, we can add a new sport without changing the core system. We only need to create a new plugin that follows the Sport API. This makes the system easier to update and expand in the future.

4.2 Example Sport (Football)

To show how the plugin structure works, we included a football implementation in the system.

The football plugin provides implementations of the interfaces defined in the Sport API. These classes define the specific rules of football, such as how matches are played, how scores are calculated, and how ties are handled.

For example, the FootballMatchSimulator class generates match results according to football rules. The FootballScoringRule determines how many points are given for a win, draw, or loss. The FootballTieBreakerRule decides how teams are ranked when they have the same number of points.

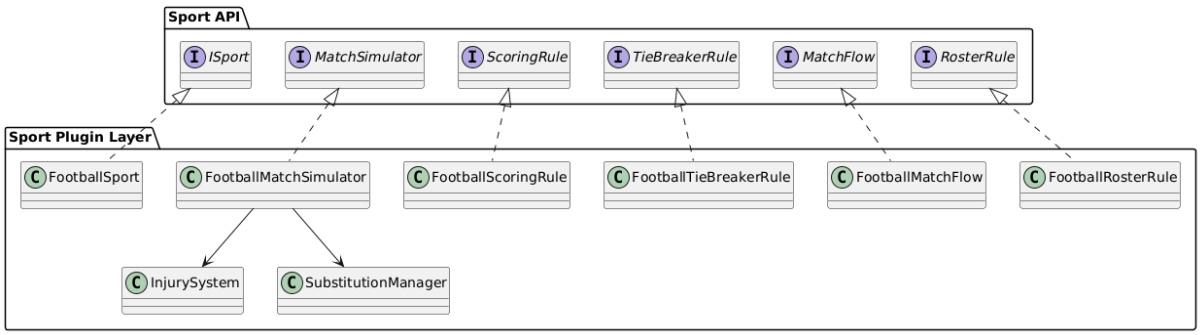

Figure 4.1: Sport API

5 GameFacade API

The GameFacade provides a simple way to interact with the core system. It works as the main connection between the UI layer and the rest of the system.

Instead of directly using domain classes like League, Team, or Match, the UI sends requests to the GameFacade. The facade then calls the necessary manager classes and domain objects to complete the operation. This hides the internal details of the system from the UI.

This structure helps reduce direct dependency between the UI and the core system. It also makes the system easier to manage because all external operations go through a single, clear interface. Some example operations of the GameFacade are creating a league, adding teams, simulating matches, and getting the current league standings.

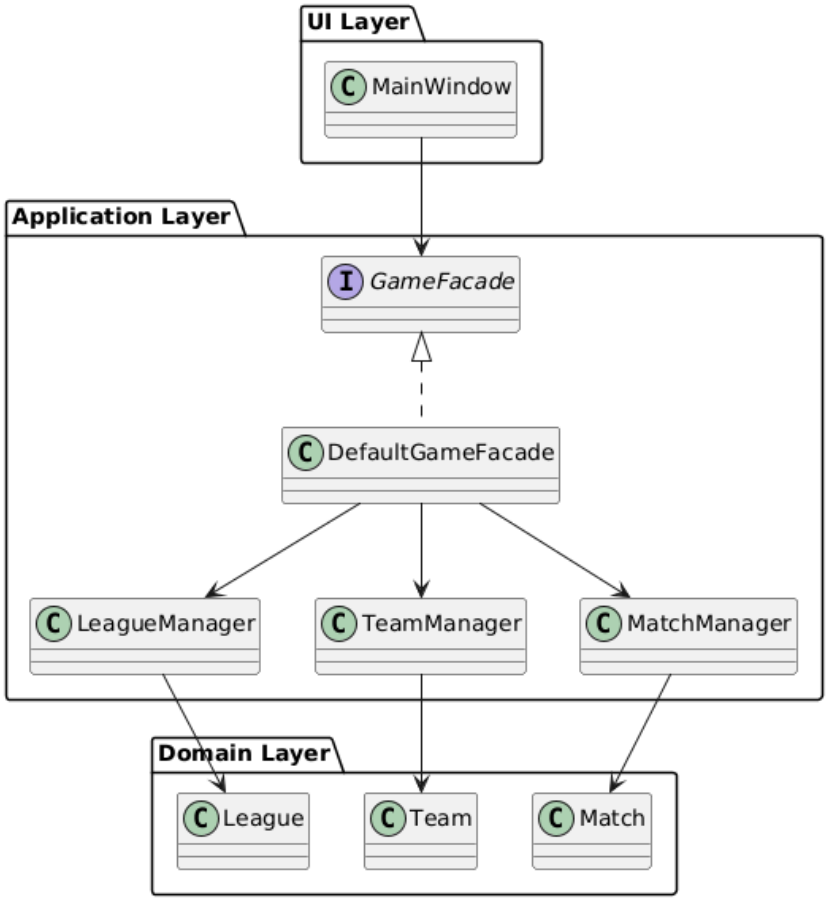

Figure 5.1: GameFacade API

6 Conclusion

In this project, we designed a flexible and extensible Sports Manager system based on a layered architecture. The main goal was to create a structure that is not dependent on a single sport and that can support new sports in the future without changing the core system.

By separating the system into UI, Application, Domain, and Sport Plugin layers, we ensured that each part has a clear responsibility. The domain layer contains the core entities of the system and remains independent from sport-specific rules. Sport-related behaviors are handled through the Sport Plugin architecture, which allows each sport to define its own rules while following a common interface.

The Football implementation was included as an example to demonstrate how the plugin structure works. Since the system depends only on abstract interfaces, new sports can be added easily by implementing the same structure.

Overall, this design makes the system more organized, modular, and easier to extend. The architecture supports future development and allows the system to grow without affecting the existing components.